Project Requirements for Supervised Machine Learning - 365 Coffee Café

Overview:

Welcome to the analytics team. 365 Coffee Café operates three busy outlets in Karachi. Since January 2025, every till transaction has been recorded together with customer, promotion, order-channel and service-quality information. We have plenty of data, but we still make retention decisions by instinct.

That needs to change. Management wants to know which visits are likely to turn into another visit within the next fourteen days. If we can identify the customers most likely to return - and the behaviours associated with repeat visits - we can improve promotions, loyalty offers and service decisions without treating every customer the same.

This is a supervised learning classification assignment. You are not being asked to try every algorithm on the internet. I want one defensible Random Forest classification workflow, built carefully from the data you were given, evaluated honestly, and translated into decisions the café can act on.

What matters most is your reasoning. A model score by itself tells me very little. I want to see how you changed transaction-line data into the correct modelling grain, why you selected each feature, what you deliberately left out, and what the business should do differently after seeing the results.

1. **What You Are Working With**

   - **Dataset:** 365 Coffee Cafe Dataset.xlsx. Use the supplied workbook only. Do not replace it with Kaggle, UCI or another dataset because the requirements below are built around this business structure.
   - **Scale:** 149,112 transaction lines across 31 fields, covering 1 January to 30 June 2025.
   - **Workbook structure:** Dataset contains the transaction records and Data Dictionary explains every field. Read the dictionary before writing modelling code.
   - **Important grain issue:** one row is an item sold, not one customer visit. Several rows can belong to the same basket_id. Your prediction target exists at visit level, so you will have to reorganise the raw data before modelling.
   - Treat the file as confidential commercial data. Keep it within your own project folder and submission.

2. **The Business Question I Need Answered**

Your single modelling question is:

   - **Which customer visits are likely to result in a return within 14 days?** Build a model that predicts returned_within_14_days from information that would be available at the end of the current visit.

The answer should help management decide where loyalty effort is worth spending. A useful submission will not stop at Yes/No predictions; it will explain what characteristics are associated with returning customers and which of those characteristics the café can realistically influence.



3. **Getting to Know the Data**

- Report the row count, column count and date range before changing the data.
- Describe the important variables and separate numerical, categorical, identifier and target fields. Use the Data Dictionary rather than guessing from values.
- Run missing-value and duplicate checks and print the actual counts. Do not write "no missing values" without showing how you checked.
- Show the basic business picture before modelling: revenue by branch, sales by product category, sales by hour and weekday, average wait time, average service rating and the overall return-within-14-days rate.
- Write one or two sentences under every major chart explaining what the result means to café management. An unlabelled chart with no interpretation is decoration, not analysis.
- Three things to watch before you model:
  - **Transaction lines are not visits.** If one basket contains three products, that is still one visit. Your modelling table must contain one row per basket_id.
  - **Future information is leakage.** Do not create a feature using events that happen after the visit you are trying to predict. The model is supposed to make the decision at that point in time, not after the customer has already returned.
  - **Customer history must be used carefully.** visit_number and days_since_previous_visit are already backward-looking. Do not calculate "total future visits" or full-period customer totals and then use those to predict an earlier visit.

4. **Preparing the Modelling Table**

I expect you to build the visit-level dataset yourself. This is the most important data-science step in the assignment.

- Group transaction lines by basket_id so that each basket becomes one observation.
- Keep visit-level information such as branch, date/time, customer characteristics, loyalty tier, order channel, payment method, promotion information, wait time and service rating where appropriate.
- Aggregate item-level values sensibly. For example, transaction quantities and revenue can be summed, while rating or wait-time fields should not be blindly summed. State the aggregation rule you chose for each engineered field.
- Use returned_within_14_days as the target and confirm that the target is consistent within each basket before collapsing the rows.
- Check the final class balance and report the percentage of visits in each class.
- Encode categorical columns where required. Use the encoding methods you have already practised in class and document the resulting features.



- Split the visit-level data into training and test sets. State the split ratio, use a fixed random seed and stratify on the target so that both sets preserve the class balance.
- Fit any preprocessing step only where appropriate and avoid learning information from the test set before evaluation.

5. **Model to Build**

Use RandomForestClassifier as the required model for this project. The purpose is not to compare algorithms you have not been taught. The purpose is to prove that you can take a real operational dataset, prepare it correctly and apply the classification workflow you already know.

- Train the Random Forest on the training set only and generate predictions for the held-out test set.
- Explain why this is a classification problem, why the target is binary, and why Random Forest is a reasonable model for mixed business features.
- List the features used in the final model and give a short reason for including each group of features.
- List at least three fields you deliberately excluded and explain why - for example identifiers, duplicated information or leakage-prone fields.
- Fix random_state so that another analyst can reproduce your result.

6. **Training and Evaluation**

- Show the Confusion Matrix and report Accuracy, Precision, Recall and F1-Score on the test set.
- Report ROC-AUC and explain in plain English what a higher ROC-AUC means in this business problem.
- Calculate the score a lazy model would get by always predicting the majority class, and show that your model performs meaningfully better than that baseline.
- Do not choose a metric just because it is the largest number. Explain which mistake matters more to management: missing a customer who would return, or spending retention effort on someone who would not.
- Show at least one table or chart comparing actual outcomes with model predictions. If the model performs poorly for a particular branch, loyalty tier or order channel, point that out.

7. **What You Tell the Business**

- Identify the behaviours or customer characteristics most strongly associated with repeat visits.
- Separate factors management can influence from factors it cannot. For example, order channel or wait time can lead to operational action; customer age may be descriptive but is not a retention lever by itself.
- Comment on wait time, service rating, promotion/discount usage, branch differences and loyalty tier where the data supports a conclusion.



- Give at least three specific recommendations. Each recommendation must be connected to a number, chart or model result from your own analysis.
- State the limitations honestly. This dataset does not contain cost/margin data, staff rosters or weather, so do not make causal or profitability claims that the file cannot support.

8. **Optional Streamlit Application**

For students who want to take the project through the full workflow, build a simple Streamlit page that accepts the final model inputs and returns:

- Predicted class: likely to return / unlikely to return.
- Model probability for the positive class, displayed as a percentage.
- A short note reminding the user that the score is a decision-support estimate, not a guarantee.

This application is not required for the core modelling marks, but a working deployment can be used as portfolio evidence.

9. **Marking**

- **Data understanding and EDA - 15 Marks.** Correct grain, data checks, relevant charts and written interpretation.
- **Visit-level data preparation - 20 Marks.** Correct basket aggregation, target consistency, leakage avoided and preprocessing justified.
- **Feature selection and encoding - 10 Marks.** Defensible feature set; identifiers and leakage-prone fields excluded.
- **Random Forest modelling - 20 Marks.** Correct train/test workflow, reproducible model and clear modelling rationale.
- **Evaluation - 20 Marks.** Confusion Matrix, Accuracy, Precision, Recall, F1, ROC-AUC and majority-class baseline interpreted correctly.
- **Business insights and conclusion - 15 Marks.** Actionable recommendations supported by evidence and honest limitations.
- **Total: 100 Marks.**

10. **Knowledge to Acquire from Google / ChatGPT**

Around 95 percent of this project comes directly from techniques already covered in class. I am deliberately leaving the following small items for you to research independently because that is part of working as an analyst:

- **Majority-class baseline -** learn how to calculate the accuracy obtained if a model predicts only the most common class, and why this is a useful sanity check.



- **Random Forest feature importance -** learn how feature_importances_ can be extracted and displayed, and what its limitations are. Use it only as supporting evidence, not proof of causation.
- **Practical leakage checks -** research examples of target leakage in customer-retention problems so that you can defend why every feature in your model would genuinely be known at prediction time.

Important: You may use Google or ChatGPT to understand the topics above, find syntax examples and troubleshoot your own code. You may not ask for or submit a complete ready-made solution. Any code you use must be adapted to your dataset, and you must be able to explain it line by line during review.

How I Want the Work Delivered:

- One Jupyter notebook (.ipynb), cleanly run from top to bottom with all outputs visible.
- Markdown commentary throughout. Explain decisions and reasoning, not Python syntax.
- The visit-level modelling dataset you produced from the raw workbook.
- A written Project Report (PDF) covering the problem, approach, results, business recommendations and limitations.
- If you complete the optional Streamlit app, include the source code and live link.

A Few Ground Rules:

- This is individual work. Discussing concepts is fine; sharing notebooks or copying complete solutions is not.
- Google and ChatGPT may be used for the research items in Section 10 and for debugging your own work. They may not be used to generate a complete project that you submit without understanding.
- The reasoning behind a modelling decision carries as much weight as the code that implements it.

This is the kind of work a commercial analytics team does when it tries to turn transaction logs into customer decisions. Make the notebook easy for another analyst to follow and make the final explanation useful to someone running the cafés, not only someone who knows machine learning.

Kind regards,

Muhammad Anas

Manager, Data Science & AI,

365 Coffee Café Inc.